

**Role:** You are a senior full-stack developer and UI/UX expert specializing in React, Tailwind CSS, and modern backend integration.

**Objective:** Build a modern, sleek web application for private pet management called "מחמד". Project folder is PetControl.

### Core Technical Requirements:

1. **Localization:** The entire UI must be in Hebrew with full RTL (Right-to-Left) support using Tailwind CSS (`dir="rtl"`).
2. **Authentication:** Implement mandatory Google ID Authentication. Ensure users can only access, view, and manage their own private pet data.
3. **Data Model:** \* **Pet Types:** Dogs and Cats.
  - **Pet Profile:** Name, Date of Birth, Breed/Race (including support for "Mixed/מערב"), Profile Photo (Cloudinary or S3 integration), Home Address, and Food Allergies.
  - **Medical Records:** A nested collection for each pet to log Vet Visits, including categories for "Routine Check," "Vaccine," and "Special Treatment."
4. **Special Integration:** \* Include a field for "Chip ID."
  - Implement a feature/link to lookup the Chip ID in the **Israeli National Dog Registration database** (המאגר הארצי לרישום כלבים).  
URL: [המאגר הארצי לרישום כלבים וחובת שיבוב - מידע כללי ושירותים משרד החקלאות וביטחון המזון](#).
  - Provide an easy-to-use interface to verify if the chip is registered correctly via the Ministry of Agriculture's public portal.
5. **UI/UX:** Use a "Modern & Sleek" aesthetic. Prioritize clean white space, rounded corners (using rounded-2xl), and a soft color palette (e.g., indigo or teal).

**Initial Task:** 1. Propose the tech stack (e.g., Next.js, Firebase/Supabase for Auth/DB). 2. Outline the database schema for Users, Pets, and Medical Logs. 3. Scaffold the main dashboard where a user sees their pets in a grid format.

---

### Key Tips for Success

- **Database Choice:** Since you want private data management and Google Auth, **Supabase** or **Firebase** are excellent choices because they handle Row Level

Security (RLS) natively. This ensures User A can never see User B's pets at the database level.

- **The Israeli Database:** The Ministry of Agriculture doesn't have a public open API for the dog registry, so your app will likely need to perform a "URL Redirect" with the chip ID as a query parameter or use a simple web-scraping utility if you want to pull the data directly into your UI.
- **RTL Design:** use logical properties in CSS (like ps-4 instead of pl-4) to ensure the layout doesn't break if you ever decide to support English later.

## 1. Database Schema (Supabase/PostgreSQL Style)

Using a relational structure is best here because of the "Medical Records" requirement. This ensures that every vaccine or checkup is strictly tied to a specific pet.

SQL

-- 1. Users Table (Handled by Google Auth, but we link a profile)

create table profiles (

id uuid references auth.users on delete cascade primary key,

full\_name text,

avatar\_url text,

updated\_at timestamp with time zone

);

-- 2. Pets Table

create table pets (

id uuid default uuid\_generate\_v4() primary key,

owner\_id uuid references profiles(id) not null,

name text not null,

type text check (type in ('dog', 'cat')),

dob date,

breed text, -- 'Mixed' or specific breed

```
photo_url text,  
home_address text,  
allergies text,  
chip_id text unique,  
created_at timestamp with time zone default now()  
);
```

-- 3. Medical Records Table

```
create table medical_records (  
  id uuid default uuid_generate_v4() primary key,  
  pet_id uuid references pets(id) on delete cascade,  
  visit_date date not null,  
  visit_type text check (visit_type in ('routine', 'vaccine', 'treatment')),  
  description text,  
  vet_name text,  
  created_at timestamp with time zone default now()  
);
```

-- CRITICAL: Row Level Security (RLS)

-- This ensures Eran only sees Eran's pets.

```
alter table pets enable row level security;  
create policy "Users can only see their own pets"  
  on pets for all  
  using (auth.uid() = owner_id);
```

---

## 2. The "Modern & Sleek" RTL Component

For the UI, we'll use **Tailwind CSS** with logical properties (like space-x-reverse and text-right). This ensures that if you ever add English, the layout doesn't break.

### Key Hebrew UI Elements:

- **Font:** Heebo or Assistant (Google Fonts) are the gold standard for modern Israeli tech apps.
- **Direction:** We use `dir="rtl"` on the wrapper.

TypeScript

// PetCard.tsx

```
import React from 'react';
```

```
const PetCard = ({ pet }) => {
```

```
  return (
```

```
    <div className="max-w-sm rounded-2xl overflow-hidden shadow-lg bg-white border border-slate-100 transition-hover hover:shadow-xl" dir="rtl">
```

```
      {/* Profile Photo */}
```

```
      <div className="relative h-48 w-full bg-slate-200">
```

```
        <img
```

```
          src={pet.photo_url || '/placeholder-pet.jpg'}
```

```
          alt={pet.name}
```

```
          className="w-full h-full object-cover"
```

```
        />
```

```
        <span className="absolute top-3 right-3 bg-white/90 px-3 py-1 rounded-full text-xs font-bold text-indigo-600">
```

```
          {pet.type === 'dog' ? 'חתול' : 'כלב'}
```

```
        </span>
```

```
      </div>
```

```
<div className="p-5 text-right">

  <h3 className="text-2xl font-bold text-slate-800 mb-1">{pet.name}</h3>

  <p className="text-sm text-slate-500 mb-4">{pet.breed} | {pet.dob}</p>

  <div className="space-y-2">

    <div className="flex items-center text-sm text-slate-600">

      <span className="font-semibold ml-2 text-indigo-500">מספר שבב:</span>

      <span>{pet.chip_id || 'לא הוזן'}</span>

    </div>

    { /* Action Button: Israeli Registry Lookup */ }

    <a

      href={` https://www.moag.gov.il/yechidot/vetserv/dog_registry/Pages/default.aspx?chip=${
pet.chip_id}`}

      target="_blank"

      className="mt-4 block text-center w-full py-2 bg-slate-100 hover:bg-indigo-50 text-
indigo-700 rounded-xl font-medium transition-colors"

      >

        בדיקה במאגר הארצי 🔍

      </a>

    </div>

  </div>

</div>

);

};
```

export default PetCard;

---

### 3. Pro-Tip: Handling the Israeli National Registry

Since there is no official JSON API for the Ministry of Agriculture's dog database, the best "modern" UX is to create a **Search Shortcut**.

When the user clicks the "Lookup" button in your app, you can pass the chip ID directly into their search form if their URL structure allows it, or simply provide a "Copy Chip ID" button followed by a link to the [Ministry's portal](#).

**Task:** Create a robust "Add New Pet" form component using React, Tailwind CSS, and react-hook-form with zod for validation.

#### Requirements:

1. **Layout:** \* Full RTL support (dir="rtl").
  - Use a "Step-by-Step" or a clearly sectioned "Card" layout to keep it sleek.
  - Labels must be aligned right, and inputs should have icons (e.g., a dog icon for name, a calendar for DOB).
2. **Form Fields & Specific Logic:**
  - **Pet Name (שם החיה):** Required text field.
  - **Type (סוג):** A toggle or radio group between "כלב" (Dog) and "חתול" (Cat).
  - **Breed (גזע):** A text input with a "מעורב" (Mixed) checkbox that disables the text input if checked.
  - **Chip ID (מספר שבב):** 15-digit numeric field. Validate that it is exactly 15 digits (standard for ISO chips).
  - **Date of Birth (תאריך לידה):** Date picker. Validation: Cannot be in the future.
  - **Allergies (אלרגיות למזון):** Text area.
  - **Address (כתובת מגורים):** Text field (integrating Google Maps Autocomplete is a plus if possible).
3. **Hebrew Validation (Zod Schema):** Implement a Zod schema with custom Hebrew error messages. Examples:
  - required\_error: "שדה זה הוא חובה"

- invalid\_type\_error: "נא להזין מספרים בלבד"
- min: "השם חייב להכיל לפחות 2 תווים"

4. **Photo Upload:** Include a "Click to upload" area with a preview of the pet's photo before saving.

**Output:** Provide the React component code and the Zod schema definition. Ensure all Tailwind classes use logical properties (e.g., mr-2 becomes ms-2).